How the backend reads Gmail

The calls the dashboard backend makes to read lead replies from the connected Gmail inbox, step by step. Marketing Agent dashboard, October 2026.

The code is in two files: `dashboardbackend/app/gmail.py` talks to Google, and `dashboardbackend/app/inbox.py` decides what to read and what to keep.

1. Getting permission to read

The backend never sees the Gmail password. It uses a key Google gave when the admin connected Gmail.

- **Connecting (done once, by the dashboard):** the admin clicked **Connect Gmail** on History. Google showed its consent screen with **read-only** permission (`gmail.readonly`) and gave back a **refresh token**, a long-lived key.
- **Storing it safely:** the dashboard's server encrypts the refresh token (AES-256-GCM, with a key derived from `GOOGLE_CLIENT_SECRET`) and saves it in the `gmail_connection` table. A copy of the database alone cannot read the mail.
- **When a check starts,** the backend reads that row and decrypts the refresh token with the same secret (that is why it needs `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET`, the same values as the dashboard), sends it to Google (`oauth2.googleapis.com/token`) and gets a short-lived **access token**, valid about an hour. It keeps that token in memory and reuses it until shortly before it expires.

Because the permission is read-only, Google itself blocks sending, deleting or changing mail, even if the code tried.

2. Asking Gmail which messages exist

```
GET https://gmail.googleapis.com/gmail/v1/users/me/messages
    ?q=in:inbox after:<date>&maxResults=500
```

- `in:inbox`: only mail in the Inbox.
- `after:<date>`: from the day before the **first outreach email** onwards; no reply can be older.
- Gmail returns only **ids**, up to 500 per page. The backend follows `nextPageToken` for up to 20 pages: at most 10,000 ids per check.

3. Choosing what to read

- It removes ids already in `inbox_seen` (the “already looked at” list). Exception: a reply whose saved copy was deleted is read again.
- It keeps up to **25 new messages**, **oldest first**. The rest wait for the next check, and the page says “N more emails will be read on the next check”.

4. Reading each message

```
GET https://gmail.googleapis.com/gmail/v1/users/me/messages/<id>?format=full
```

- 5 messages are fetched at a time, to stay fast without hitting Gmail's limits.
- A message deleted since it was listed is simply skipped.

Parsing (`parse()` in `gmail.py`) turns Google's format into plain fields:

Field	Where it comes from
from_email	the From header, address only, lower-cased (“Priya <Priya@Acme.co>” → priya@acme.co)

Field	Where it comes from
subject, message_id, in_reply_to, references	the headers with those names
thread_id, labels	Gmail's conversation id and labels (INBOX, SENT, ...)
received_at	when Gmail received it
text	the plain-text part of the email, or the HTML part turned into text if there is no plain one; cut at 20,000 characters

HTML is never shown or run: tags, scripts and styles are stripped and only the text is kept, so a message cannot run code or load trackers. **Attachments are never opened.**

5. Deciding what it is and saving it

Message	Result
From the connected address, or labelled SENT	**Our own mail**: skipped
From mailer-daemon or postmaster	**Bounce**: saved, marked bounced
Matched to one of our emails: headers, then thread, then sender address	**Reply**: saved
Out-of-office (headers such as Auto-Submitted, or the subject)	Saved but hidden
Anything else	**Not a reply**

Every message gets a row in `inbox_seen`. Replies also go into `inbox_replies`: the sender, text, date, matched lead and how it was matched.

6. Two extra passes on every check

- **Look again:** mail judged “not a reply” in the last 14 days is read again (up to 25) if one of our emails was added after it was first seen. This catches a reply that arrived before its lead was in the database.
- **Did we answer?** For up to 10 unanswered replies, it fetches the thread with only basic details (`/threads/<id>?format=metadata`). If a newer message there is labelled SENT, it records **We replied** and the date.

7. When something goes wrong

The page shows a clear message instead of signing the user out:

Problem	Message
Gmail not connected	“Gmail is not connected. An admin can connect it on the History page.”
Google ended the access (password change, access removed, or the 7-day limit while the Google app is in Testing)	“… An admin can connect Gmail again on the History page.”
Gmail API not enabled in Google Cloud	Says exactly where to enable it
Too many requests	“Gmail is limiting requests. Wait a minute and try again.”
Google did not answer	“Try again in a minute.”

In one line

Decrypt the saved key → get an hour-long token → list Inbox ids since the first outreach → skip ones already seen → fetch up to 25 new ones in full → turn each into plain text and headers → classify, match and save → note what was seen.

The 7-day limit matters for real use: while the Google Cloud app is in **Testing** mode, Google ends the access after 7 days and an admin has to click Connect Gmail again. Publishing the app (or using a Google Workspace “Internal” app) removes that limit.

Where it lives in the code

What	File
Decrypt the key, get the token, Gmail API calls, parse a message	dashboardbackend/app/gmail.py
List, choose, fetch, classify, match, save, look again, did we answer	dashboardbackend/app/inbox.py
Bounce and out-of-office rules, Message-ID parsing	dashboardbackend/app/reply_rules.py
Connect Gmail, encrypt and save the key	dashboard/core/server/gmail.js, gmailTokens.js
What happens after (score and groups)	docs/how-positive-leads-works.pdf